



FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065 (INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

1

RANDOM WALK SIMULATION USING AGENTPY

Student Name	Justin D. Vergara
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	October 2, 2026

I. BACKGROUND INFORMATION

A **random walk** is a mathematical process that describes a path consisting of a sequence of **random steps**. Random walks are used in many fields, such as physics, biology, economics, and computer science, to model systems that evolve randomly over time.

In this laboratory, we simulate multiple agents moving randomly on a grid using **AgentPy**, a Python library for **agent-based modeling**. Each agent represents an entity that can move one step at a time in a random direction. This simulation helps students understand:

- How **agents move and interact** in a grid-based environment.
- The concept of **bounded movement**, ensuring agents stay within the grid limits.
- What changing **parameters** like the number of agents, grid size, or number of steps affects the system.
- How **agent-based models** can be used to simulate real-world phenomena, such as **diffusion, crowd dynamics, or swarm behavior**.

II. TECHNICAL ASSESSMENT

RANDOM WALK SIMULATION USING AGENTPY

Objective

At the end of this laboratory activity, students are expected to:

1. To understand the concept of **random walks** in a grid environment.
2. To implement a **multi-agent simulation** using Python and AgentPy.
3. To visualize the **movement of agents** on a grid over time.

Materials Needed

- Computer with Python 3.x installed
- Libraries: agentpy, matplotlib
- Code editor (e.g., VSCode, PyCharm, or Jupyter Notebook)

General Instructions

1. This **Technical Assessment (TA4)** must be completed individually.
2. Follow the Task below.
3. Include clear comments in your code to describe its logic.
4. Each program should display the correct input, process, and output.
5. Ensure that your script displays clear and readable outputs in the Command Window.
6. Compile all Python codes and corresponding outputs into one PDF file.
7. Save it as **TA1_lastname.pdf**

(Example: TA1_Patilano.pdf)

8. Submit the **python file** and **PDF file**.

Activity Instructions

Step 1: Run the Code

1. Open the provided Python code.

Random Walk Simulation (Console-Based)

```
import agentpy as ap
import random
import matplotlib.pyplot as plt
import matplotlib.animation as animation

# ----- Get user input from console -----
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size (e.g., 10 for 10x10): "))
num_steps = int(input("Enter number of steps: "))

# ----- Agent Definition -----
class RandomWalker(ap.Agent):
    def step(self):
        direction = random.choice([(1,0), (-1,0), (0,1), (0,-1)])
        x, y = self.position
        # Keep agents inside grid
        x = max(0, min(self.model.p.grid_size[0]-1, x + direction[0]))
        y = max(0, min(self.model.p.grid_size[1]-1, y + direction[1]))
        self.position = (x, y)

# ----- Model Definition -----
class RandomWalkModel(ap.Model):
    def setup(self):
        # Create agents
        self.agents = ap.AgentList(self, self.p.agents, RandomWalker)

        # Initialize random positions
        for agent in self.agents:
            agent.position = (random.randint(0, self.p.grid_size[0]-1),
                             random.randint(0, self.p.grid_size[1]-1))

        # Create grid (optional)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        for agent in self.agents:
            agent.step()

# ----- Parameters from user input -----
parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),
    'steps': num_steps
```

```

}

# ----- Run Model -----
model = RandomWalkModel(parameters)
model.setup()

# ----- Interactive Animation -----
fig, ax = plt.subplots()
ax.set_xlim(0, model.p.grid_size[0])
ax.set_ylim(0, model.p.grid_size[1])
ax.set_xticks(range(model.p.grid_size[0]+1))
ax.set_yticks(range(model.p.grid_size[1]+1))
ax.grid(True)

scat = ax.scatter([], [], s=200) # s = size of agents

def update(frame):
    model.step()
    x = [agent.position[0] for agent in model.agents]
    y = [agent.position[1] for agent in model.agents]
    scat.set_offsets(list(zip(x, y)))
    return scat,

ani = animation.FuncAnimation(fig, update, frames=model.p.steps, blit=True, repeat=False)
plt.show()

```

2. Run the program in your terminal.
3. Enter the following values when prompted:
 - **Number of agents** (e.g., 5)
 - **Grid size** (e.g., 10 for a 10x10 grid)
 - **Number of steps** (e.g., 20)

These values control the number of agents, the size of the environment, and the simulation duration.

Step 2: Understand the Agent Behavior

1. Each agent is represented by a dot in the grid.
2. Agents move **randomly** in one of four directions: up, down, left, or right.
3. Agents **cannot leave the grid**; movement is restricted within the grid boundaries.

Step 3: Observe the Simulation

1. After running, a **grid with moving agents** is displayed.
2. Each time step updates the position of all agents.
3. Observe:
 - How agents spread across the grid over time
 - Whether agents cluster in certain areas or move freely

Step 4: Experiment with Parameters

- Repeat the simulation with **different values**:

- **More agents** (e.g., 10 or 20)
 - **Smaller/larger grid sizes**
 - **Longer simulations** (e.g., 50 steps)
- Observe and record how the **agent movement patterns change**.

Laboratory Tasks

1. **Track agent paths:** Modify the code to **record and display the path of each agent** during the simulation.
2. **Change movement rules:** Make agents **avoid each other** or move in a **preferred direction** and observe the resulting patterns.
3. **Analyze final positions:** Record the **final positions** of all agents and analyze their **distribution**.
4. **Compare scenarios:** Run **two simulations with different parameters** and compare the movement patterns.

Modified (Updated Code - Laboratory Tasks) ↴

```
import random
import agentpy as ap
import matplotlib.pyplot as plt

# 1. AGENT CLASS DEFINITION
class SmartWalker(ap.Agent):
    """
    Agent with trajectory tracking and modified directional movement
    rules.
    """
    def setup(self):
        # Initialize agent at a random coordinate within grid bounds
        self.position = (
            random.randint(0, self.model.p.grid_size[0] - 1),
            random.randint(0, self.model.p.grid_size[1] - 1)
        )
        # Laboratory Task 1: Record and track the complete path
        self.path = [self.position]

    def step(self):
        # Laboratory Task 2: Movement rules modification
        # Biased movement preference: Higher likelihood to step right or
        up
        directions = [(1, 0), (-1, 0), (0, 1), (0, -1)]
```

```

        weights = [0.4, 0.2, 0.3, 0.1] # 40% Right, 20% Left, 30% Up, 10%
Down

        move = random.choices(directions, weights=weights, k=1)[0]

        # Bounded Movement: Keep coordinates strictly inside the grid
limits
        new_x = max(0, min(self.model.p.grid_size[0] - 1, self.position[0]
+ move[0]))
        new_y = max(0, min(self.model.p.grid_size[1] - 1, self.position[1]
+ move[1]))

        self.position = (new_x, new_y)
        self.path.append(self.position)

# 2. MODEL CLASS DEFINITION
class RandomWalkModel(ap.Model):
    """
    Model environment to manage agent population and grid boundaries.
    """
    def setup(self):
        self.agents = ap.AgentList(self, self.p.agents, SmartWalker)
        self.grid = ap.Grid(self, self.p.grid_size, torus=False)
        self.grid.add_agents(self.agents)

    def step(self):
        self.agents.step()

# 3. USER INPUTS & SIMULATION EXECUTION
# Get inputs dynamically via Colab console prompts
num_agents = int(input("Enter number of agents: "))
grid_size = int(input("Enter grid size: "))
num_steps = int(input("Enter number of steps: "))

parameters = {
    'agents': num_agents,
    'grid_size': (grid_size, grid_size),

```

```

        'steps': num_steps
    }

model = RandomWalkModel(parameters)
model.run()

# 4. VISUALIZATION: TRAJECTORY PATHS & FINAL POSITIONS
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(13, 6))

# Subplot 1: Agent Trajectory Paths (Task 1)
ax1.set_xlim(-0.5, parameters['grid_size'][0] - 0.5)
ax1.set_ylim(-0.5, parameters['grid_size'][1] - 0.5)
ax1.set_xticks(range(parameters['grid_size'][0]))
ax1.set_yticks(range(parameters['grid_size'][1]))
ax1.grid(True, linestyle="--", alpha=0.5)
ax1.set_title("Task 1: Agent Paths & Trajectories", fontsize=11,
fontweight='bold')

colors = plt.cm.tab10.colors
for i, agent in enumerate(model.agents):
    px, py = zip(*agent.path)
    c = colors[i % len(colors)]
    # Path line
    ax1.plot(px, py, linestyle='-', marker='o', markersize=3, alpha=0.7,
color=c, label=f"Agent {agent.id}")
    # Starting location (square) and Ending location (triangle)
    ax1.plot(px[0], py[0], marker='s', markersize=7, color=c)
    ax1.plot(px[-1], py[-1], marker='^', markersize=8, color=c)

ax1.legend(bbox_to_anchor=(1.03, 1), loc='upper left', fontsize='small')

# Subplot 2: Final Distribution Analysis (Task 3)
ax2.set_xlim(-0.5, parameters['grid_size'][0] - 0.5)
ax2.set_ylim(-0.5, parameters['grid_size'][1] - 0.5)
ax2.set_xticks(range(parameters['grid_size'][0]))
ax2.set_yticks(range(parameters['grid_size'][1]))
ax2.grid(True, linestyle="--", alpha=0.5)

```

```

ax2.set_title("Task 3: Final Agent Positions Distribution", fontsize=11,
fontweight='bold')

final_x = [agent.path[-1][0] for agent in model.agents]
final_y = [agent.path[-1][1] for agent in model.agents]
ax2.scatter(final_x, final_y, s=120, color='crimson', edgecolors='black',
zorder=3)

for agent in model.agents:
    fx, fy = agent.path[-1]
    ax2.annotate(f"A{agent.id}", (fx + 0.15, fy + 0.15), fontsize=9,
fontweight='bold')

plt.suptitle("CS0065 Technical Assessment 1 – Justin D. Vergara (AN41)",
fontsize=13, fontweight='bold')
plt.tight_layout()
plt.show()

# 5. CONSOLE OUTPUT: FINAL POSITION & DISPLACEMENT LOGS
print("\n" + "="*60)
print("CS0065 TA1: FINAL POSITIONS & DISPLACEMENT ANALYSIS")
print("Programmed by: Justin D. Vergara – AN41")
print("="*60)
for agent in model.agents:
    start_pos = agent.path[0]
    end_pos = agent.path[-1]
    dx = end_pos[0] - start_pos[0]
    dy = end_pos[1] - start_pos[1]
    print(f"Agent {agent.id:02d} | Start: {start_pos} -> End: {end_pos} |
Net Displacement: ({dx:+d}, {dy:+d})")
print("="*60)

```

Step 5: Answer the Questions

1. What happens if the number of agents is **very high** compared to the grid size?
2. How does increasing the number of steps affect the **spread of agents**?
3. Do the agents ever move outside the grid? Why or why not?
4. How can this simulation be used to **model real-life phenomena** (e.g., diffusion, crowd movement)?

Answer the Questions

1. When the number of agents is very high compared to the grid size, severe spatial crowding occurs, causing multiple agents to occupy the same grid coordinates and frequently accumulate along the boundaries.
2. Increasing the number of steps allows agents to explore a larger area over time, while the directional bias in the code causes them to progressively drift and gather near the top-right corner of the grid.
3. Agents never move outside the grid because the movement logic uses coordinate clamping functions that strictly enforce positions to remain between zero and the maximum grid index.
4. This simulation can model real-life phenomena such as molecular diffusion in physics, disease transmission in epidemiology, animal foraging behavior in biology, and pedestrian crowd dynamics in urban planning.